

Imports

In [2]:

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import datetime
import warnings
import matplotlib.pyplot as plt
import statsmodels.api as sm
import math
import sklearn
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf as pltacf
from statsmodels.tsa.stattools import pacf as PACF
from statsmodels.graphics.tsaplots import plot_pacf as pltpacf
from pmdarima.arima import auto_arima
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
warnings.filterwarnings('ignore')
```

In [3]:

```
!gdown --id 1uhxjs5UNSTrKvkjhJhyCY_I5Hx4ep6wX
```

Downloading...

From: https://drive.google.com/uc?id=1uhxjs5UNSTrKvkjhJhyCY_I5Hx4ep6wX

To: /content/DIVISLAB.NS.csv

100% 152k/152k [00:00<00:00, 52.2MB/s]

In [4]:

```
stock_name="DIVISLAB"
```

In [5]:

```
df_stock = pd.read_csv('DIVISLAB.NS.csv', index_col='Date')
df_stock.head()
```

Out[5]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	NI
Date									
2016-01-01	1154.0	1165.000000	1145.550049	1162.500000	100178.0	4	1	1	7791.2
2016-01-04	1161.0	1173.699951	1137.800049	1149.699951	233101.0	0	4	1	7784.6
2016-01-05	1147.0	1157.500000	1138.500000	1147.500000	302272.0	1	5	1	7741.0
2016-01-06	1149.0	1154.300049	1135.000000	1137.199951	392164.0	2	6	1	7568.2
2016-01-07	1115.0	1136.550049	1115.000000	1126.949951	316523.0	3	7	1	7601.3

ARIMA - Autoregression Intergrated Moving Average

Decomposing the data

In [6]:

```
df_stock['Close'].head()
```

Out[6]:

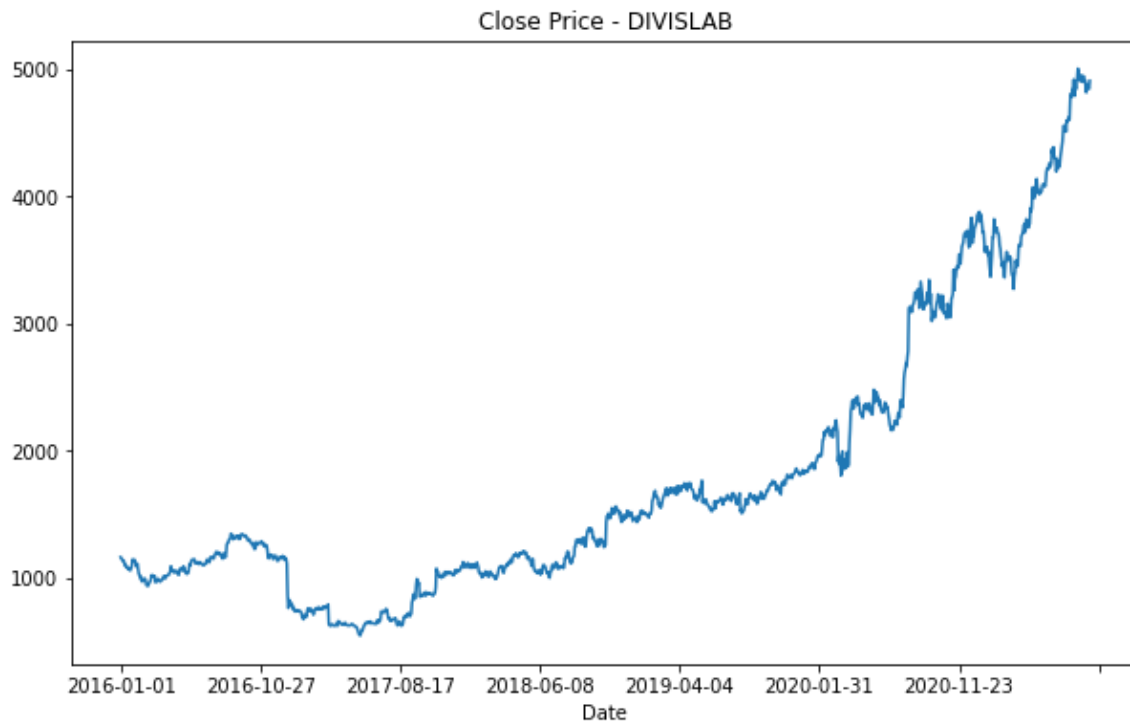
```
Date
2016-01-01    1162.500000
2016-01-04    1149.699951
2016-01-05    1147.500000
2016-01-06    1137.199951
2016-01-07    1126.949951
Name: Close, dtype: float64
```

In [7]:

```
plt.rcParams['figure.figsize'] = (10,6)
df_stock['Close'].plot()
plt.title("Close Price - "+stock_name)
```

Out[7]:

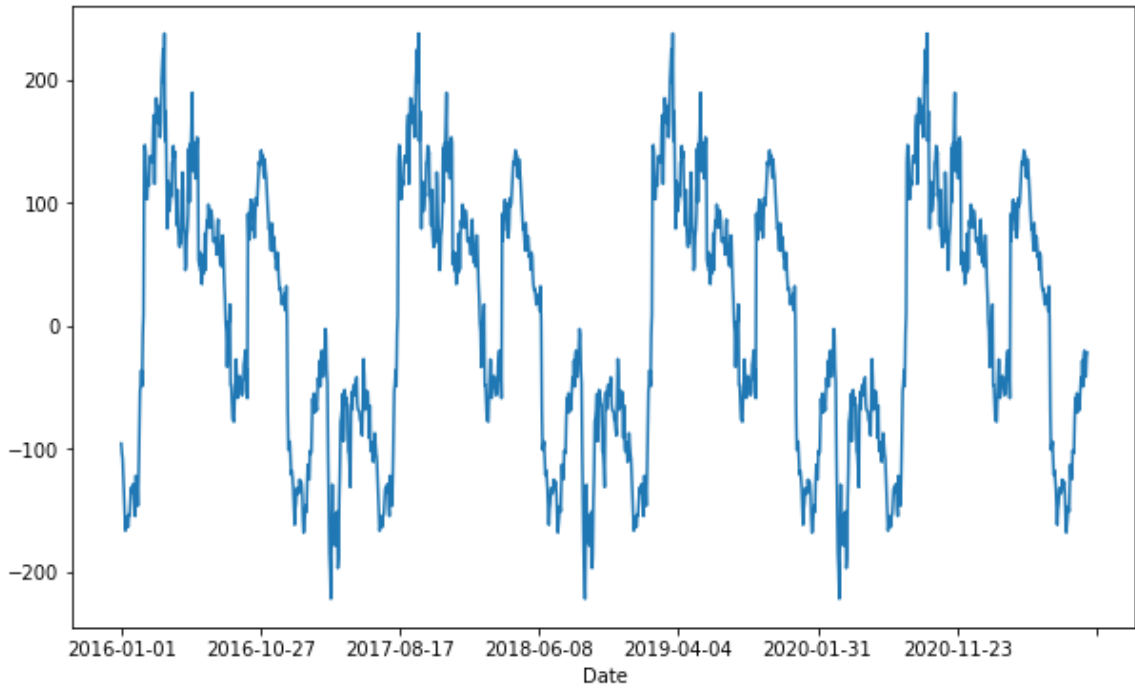
Text(0.5, 1.0, 'Close Price - DIVISLAB')



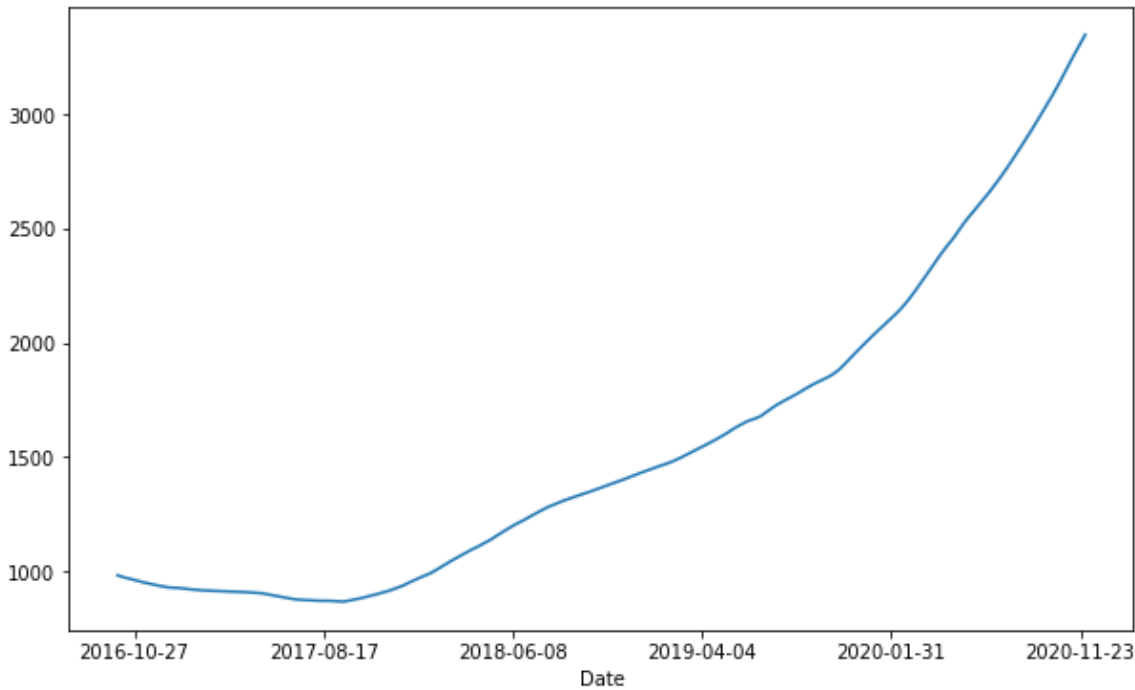
In [8]:

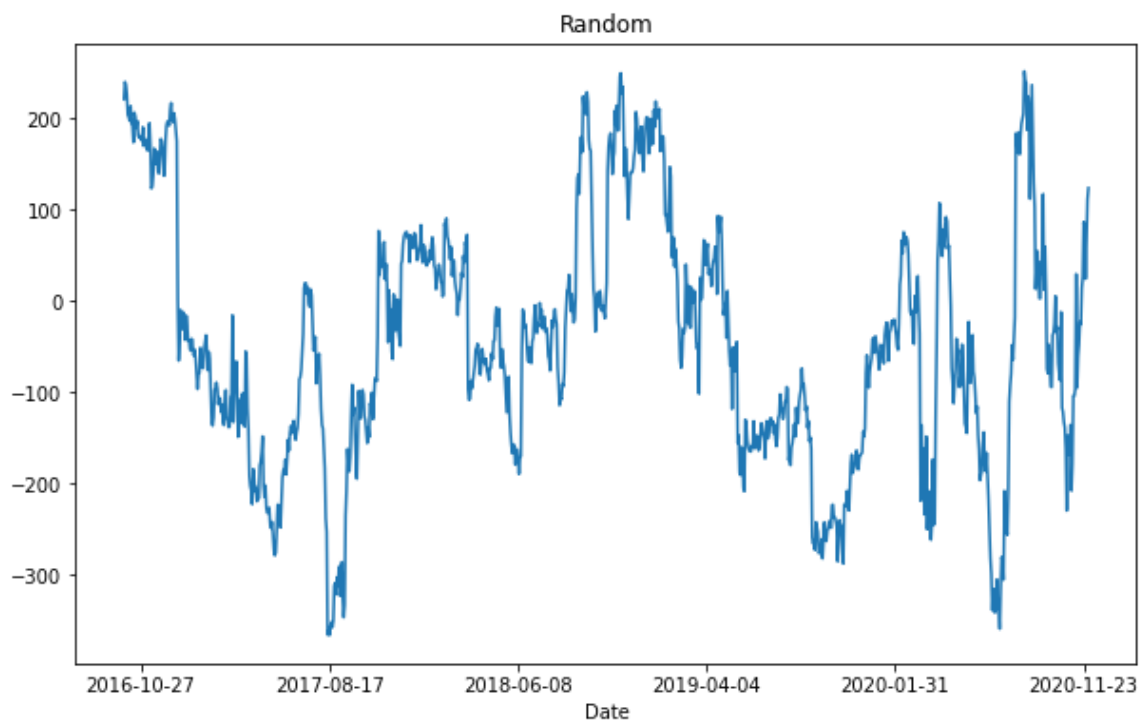
```
plt.rcParams['figure.figsize'] = (10,6)
decomposition=seasonal_decompose(df_stock['Close'],freq=365)
decomposition.seasonal.plot()
plt.title("Seasonal")
plt.show()
decomposition.trend.plot()
plt.title("Trend")
plt.show()
decomposition.resid.plot()
plt.title("Random")
plt.show()
```

Seasonal



Trend





Stationary Check

In [9]:

```

Stationary=False
dvalue=0
Stationary_df=df_stock['Close'].copy()
while(Stationary==False):
    dfctest = adfuller(Stationary_df.dropna(),autolag='AIC')
    dfoutput=pd.Series(dfctest[0:4],index=['Test Statistic','p-value','#lags used','number
of observations used'])
    for key,value in dfctest[4].items():
        dfoutput['critical value (%s)'%key]= value
    print(dfctest[1])
    if (dfctest[1]>0.05):
        Stationary=False
        Stationary_df = Stationary_df - Stationary_df.shift(1)
    else :
        Stationary=True
    dvalue+=1
print(dfoutput)
print("Differencing Value : ",dvalue)

```

```

0.9990796189377779
Test Statistic          2.626709
p-value                 0.999080
#lags used              1.000000
number of observations used 1386.000000
critical value (1%)     -3.435077
critical value (5%)     -2.863628
critical value (10%)    -2.567881
dtype: float64
Differencing Value : 1

```

ACF

In [10]:

```

from statsmodels.tsa.stattools import acf as ACF
ACFOutput = ACF(Stationary_df.dropna(),nlags=20)
print(ACFOutput)

```

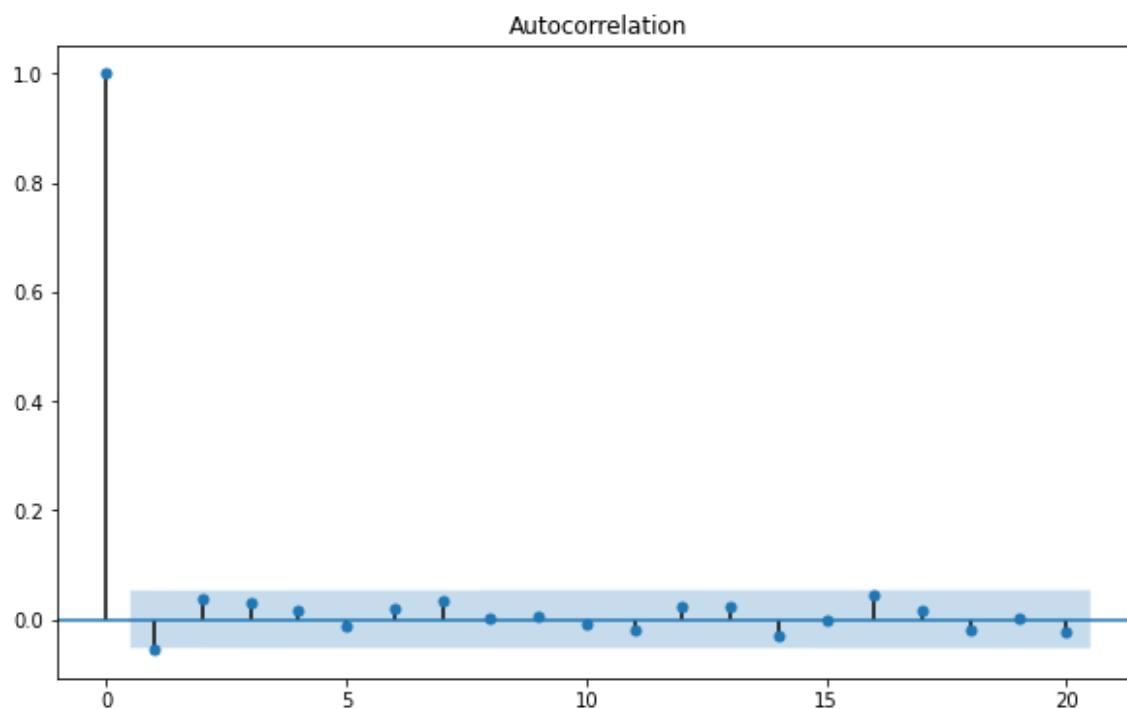
```

[ 1.00000000e+00 -5.50033366e-02  3.56929177e-02  3.05698143e-02
 1.54345112e-02 -1.35382160e-02  2.06389534e-02  3.46864291e-02
 3.60482580e-03  4.36507716e-03 -9.26130343e-03 -1.91192578e-02
 2.30102806e-02  2.19040013e-02 -2.93565969e-02 -2.26801053e-03
 4.47531034e-02  1.51396210e-02 -1.89162139e-02  6.55405844e-04
-2.35037637e-02]

```

In [11]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.acf(Stationary_df.dropna(),lags=20)
plt.show()
```



PACF

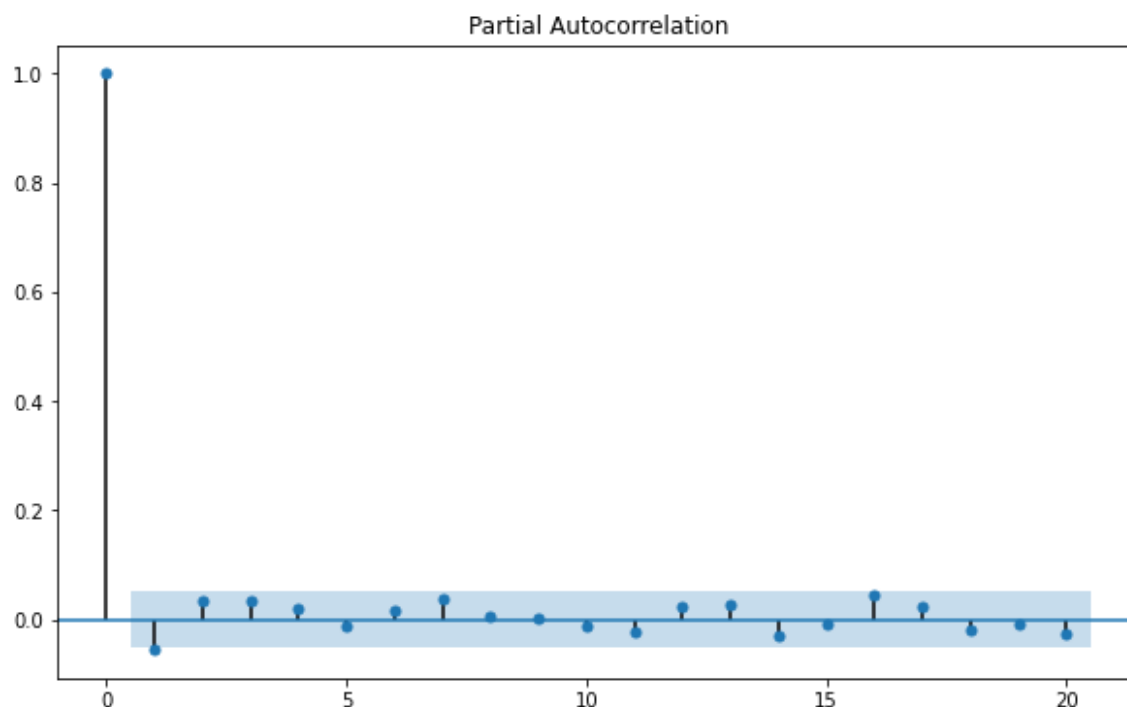
In [12]:

```
PACFOutput = PACF(Stationary_df.dropna(),nlags=20)
print(PACFOutput)
```

```
[ 1.          -0.05504302  0.03281414  0.0344867   0.01789914 -0.01414015
  0.01705806  0.03704051  0.0069156   0.00157758 -0.01263109 -0.02174405
  0.02218632  0.02566419 -0.02869371 -0.0090816   0.04464381  0.02449846
 -0.01802628 -0.00905836 -0.0267907 ]
```


In [13]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.pacf(Stationary_df.dropna(),lags=20).show()
plt.show()
```



Test and Train Split

In [14]:

```
df_stock_train=df_stock.loc['2016-01-01':'2021-07-31']['Close']
df_stock_test=df_stock.loc['2021-07-31:']['Close']
```

In [15]:

```
df_stock_train.tail()
```

Out[15]:

```
Date
2021-07-26    4921.149902
2021-07-27    4791.399902
2021-07-28    4890.600098
2021-07-29    4843.799805
2021-07-30    4906.100098
Name: Close, dtype: float64
```

In [16]:

```
df_stock_test.head()
```

Out[16]:

```
Date
2021-08-02    4928.950195
2021-08-03    5006.850098
2021-08-04    4964.250000
2021-08-05    4923.549805
2021-08-06    4916.000000
Name: Close, dtype: float64
```

Model Fitting

In [17]:

```
arima_model=auto_arima(df_stock_train,seasonal=False,max_p=2,max_d=2,max_q=2,start_p=0,
start_q=0,d=1)
```

In [18]:

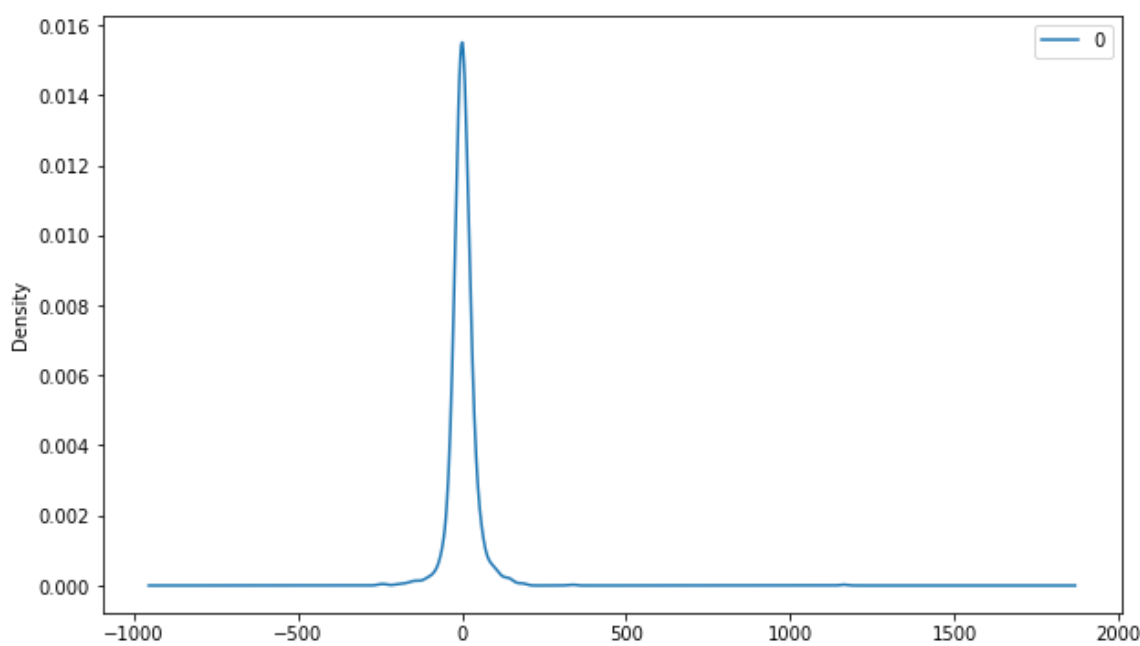
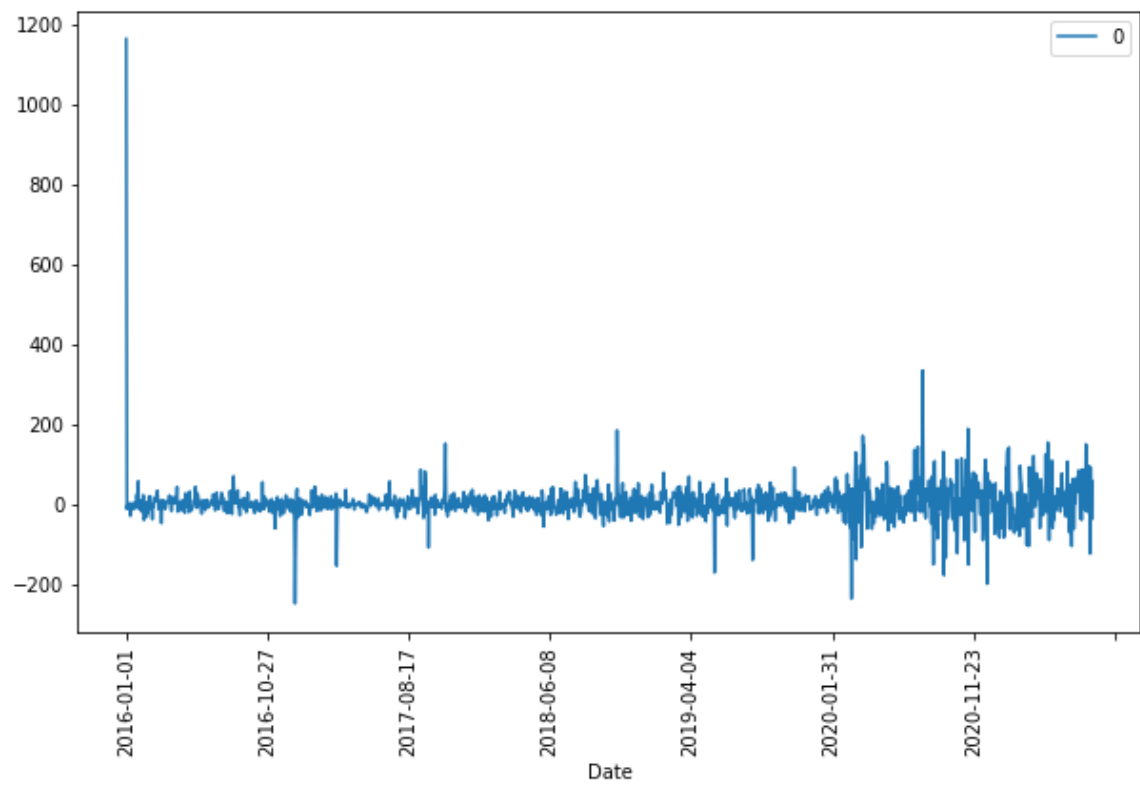
```
arima_model
```

Out[18]:

```
ARIMA(maxiter=50, method='lbfgs', order=(1, 1, 0), out_of_sample_size=0,
      scoring='mse', scoring_args={}, seasonal_order=(0, 0, 0, 0),
      start_params=None, suppress_warnings=True, trend=None,
      with_intercept=True)
```

In [19]:

```
model = ARIMA(df_stock_train, order=(1,1,1))
model_fit = model.fit()
residuals = pd.DataFrame(model_fit.resid)
plt.rcParams['figure.figsize'] = (10,6)
residuals.plot()
plt.xticks(rotation=90)
plt.show()
residuals.plot(kind='kde')
plt.show()
print(residuals.describe())
```



```
0
count    1369.000000
mean       3.679404
std       49.362535
min      -248.985918
25%      -13.446550
50%       0.800664
75%      17.183112
max      1162.500000
```

Forecasting and Testing

In [20]:

```
history = [x for x in df_stock_train.values]
predictions = list()
for t in range(len(df_stock_test.values)):
    model = ARIMA(history, order=(1,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = df_stock_test.values[t]
    history.append(obs)
error = np.sqrt(mean_squared_error(df_stock_test.values, predictions))
print('Test Error Percentage', (error/np.mean(df_stock_test.values))*100)
```

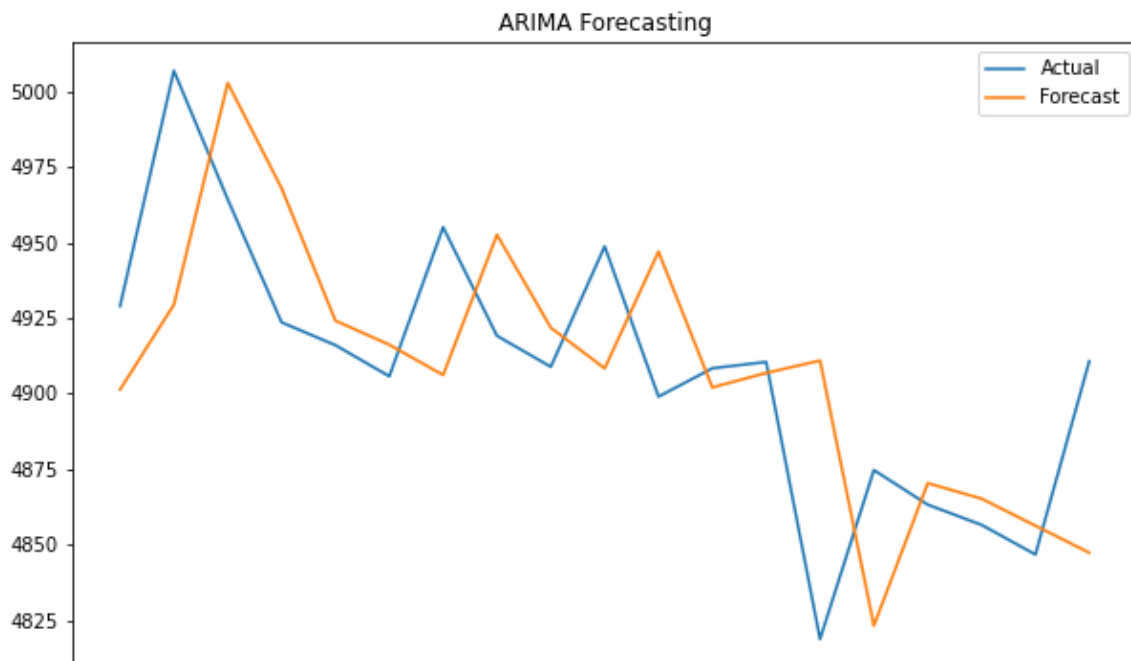
Test Error Percentage 0.854928783783886

In [21]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.plot(df_stock_test, label='Actual')
plt.plot(predictions, label='Forecast')
plt.title('ARIMA Forecasting')
plt.legend()
plt.xticks([])
```

Out[21]:

([], <a list of 0 Text major ticklabel objects>)



Forecast for August

In [22]:

```
history = [x for x in df_stock_train.values]
predictions = list()
for t in range(len(df_stock_test.values[:14])):
    model = ARIMA(history, order=(1,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = df_stock_test.values[t]
    history.append(obs)
```

In [23]:

```
predictions
```

Out[23]:

```
[4901.3095597632255,
 4929.397929522329,
 5002.835764225187,
 4967.971205990904,
 4924.1069799972265,
 4916.151286561096,
 4906.127444115527,
 4952.5495408837305,
 4921.7961288714205,
 4908.326715923364,
 4946.962252923704,
 4902.023320722064,
 4906.802860216723,
 4910.888279173102]
```

MARS - Multivariate Adaptive Regression Splines

In [24]:

```
from pyearth import Earth
```

In [25]:

```
df_stock_mars_train=df_stock.loc['2016-01-01':'2021-07-31'].iloc[:,5].copy()
df_stock_mars_test=df_stock.loc['2021-07-31:'].iloc[:,5].copy()
```

In [26]:

```
df_stock_mars_train.shape
```

Out[26]:

```
(1369, 5)
```

In [27]:

```
df_stock_mars_test.shape
```

Out[27]:

(19, 5)

In [28]:

```
df_stock_mars_train.head()
```

Out[28]:

	Open	High	Low	Close	Volume
Date					
2016-01-01	1154.0	1165.000000	1145.550049	1162.500000	100178.0
2016-01-04	1161.0	1173.699951	1137.800049	1149.699951	233101.0
2016-01-05	1147.0	1157.500000	1138.500000	1147.500000	302272.0
2016-01-06	1149.0	1154.300049	1135.000000	1137.199951	392164.0
2016-01-07	1115.0	1136.550049	1115.000000	1126.949951	316523.0

In [29]:

```
df_stock_mars_test
```

Out[29]:

	Open	High	Low	Close	Volume
Date					
2021-08-02	4893.500000	4962.149902	4893.500000	4928.950195	359231.0
2021-08-03	4930.600098	5019.899902	4900.000000	5006.850098	530586.0
2021-08-04	5017.000000	5045.200195	4954.549805	4964.250000	381205.0
2021-08-05	5000.000000	5020.000000	4916.500000	4923.549805	413085.0
2021-08-06	4950.000000	4959.000000	4884.000000	4916.000000	314260.0
2021-08-09	4980.000000	4989.799805	4831.000000	4905.700195	955464.0
2021-08-10	4930.899902	4971.600098	4895.000000	4955.049805	356348.0
2021-08-11	4954.500000	4958.950195	4840.799805	4919.100098	354008.0
2021-08-12	4929.950195	4929.950195	4872.000000	4908.850098	200081.0
2021-08-13	4900.049805	4973.000000	4860.549805	4948.700195	314044.0
2021-08-16	4945.549805	4954.350098	4870.000000	4898.950195	225012.0
2021-08-17	4893.500000	4940.000000	4875.000000	4908.350098	356466.0
2021-08-18	4911.850098	4999.049805	4902.000000	4910.450195	506142.0
2021-08-20	4861.299805	4915.399902	4757.000000	4818.700195	549942.0
2021-08-23	4849.000000	4888.700195	4788.000000	4874.649902	410817.0
2021-08-24	4890.000000	4912.149902	4792.850098	4863.200195	425873.0
2021-08-25	4863.450195	4875.000000	4818.100098	4856.500000	345800.0
2021-08-26	4855.000000	4877.200195	4829.000000	4846.700195	281990.0
2021-08-27	4846.649902	4934.049805	4824.350098	4910.700195	349435.0

In [30]:

```
x_train_mars = df_stock_mars_train.drop(['Close'], axis=1)
y_train_mars = df_stock_mars_train['Close']
x_test_mars = df_stock_mars_test.drop(['Close'], axis=1)
y_test_mars = df_stock_mars_test['Close']
```

Fitting the model

In [31]:

```
MARS_model = Earth(max_terms = 600, max_degree=3)
MARS_model_fitted = MARS_model.fit(x_train_mars, y_train_mars)
y_pred_mars = MARS_model_fitted.predict(x_test_mars[:14])
```

Evaluating the model

In [32]:

```
MSE = mean_squared_error(y_test_mars[:14], y_pred_mars)
RMSE = np.sqrt(MSE)
RMSE_mean = (RMSE/np.mean(y_test_mars[:14]))*100
print("RMSE Score :", round(RMSE, 2))
print("RMSE/Mean", round(RMSE_mean,1))
```

RMSE Score : 32.88

RMSE/Mean 0.7

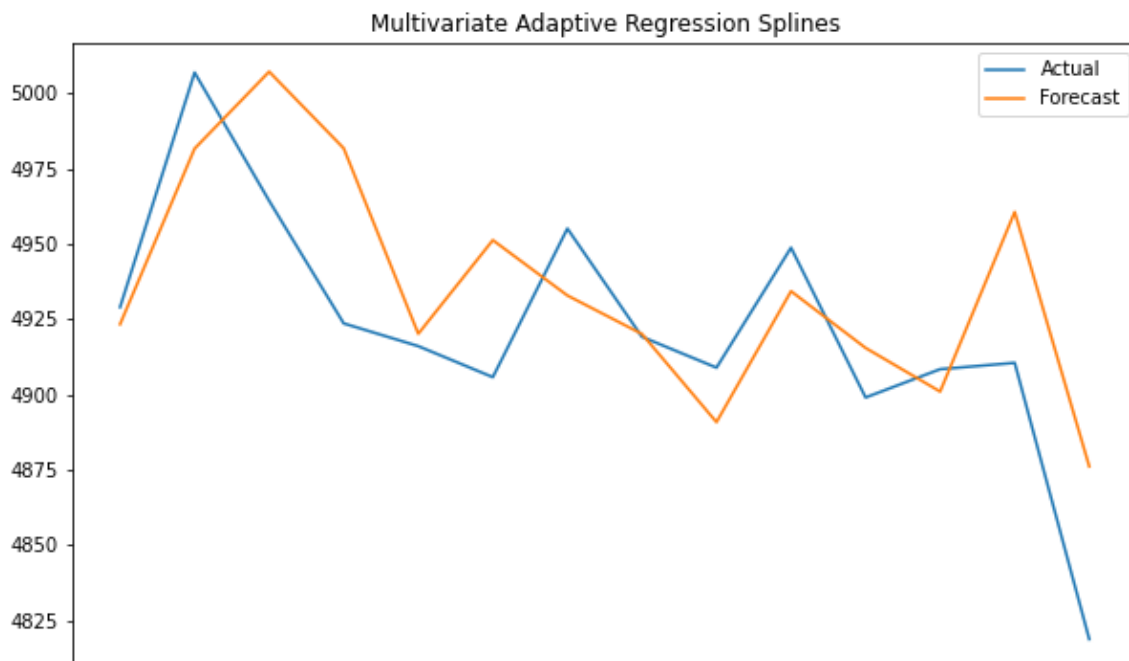
Plotting Forecast vs Actual

In [33]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.plot(y_test_mars[:14], label='Actual')
plt.plot(y_pred_mars, label='Forecast')
plt.title('Multivariate Adaptive Regression Splines')
plt.legend()
plt.xticks([])
```

Out[33]:

([], <a list of 0 Text major ticklabel objects>)



VAR - Vector Autoregression

In [34]:

```
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import adfuller
from statsmodels.tools.eval_measures import rmse, aic
```

In [35]:

```
df_stock_var = df_stock.loc['2016-01-01':'2021-08-20'].iloc[:,4].copy()
df_stock_var_train=df_stock.loc['2016-01-01':'2021-07-31'].iloc[:,4].copy()
df_stock_var_test=df_stock.loc['2021-07-31':'2021-08-20'].iloc[:,4].copy()
```

In [36]:

```
df_stock_var_train.shape
```

Out[36]:

(1369, 4)

In [37]:

```
df_stock_var_test.shape
```

Out[37]:

(14, 4)

In [38]:

```
df_stock_var.shape
```

Out[38]:

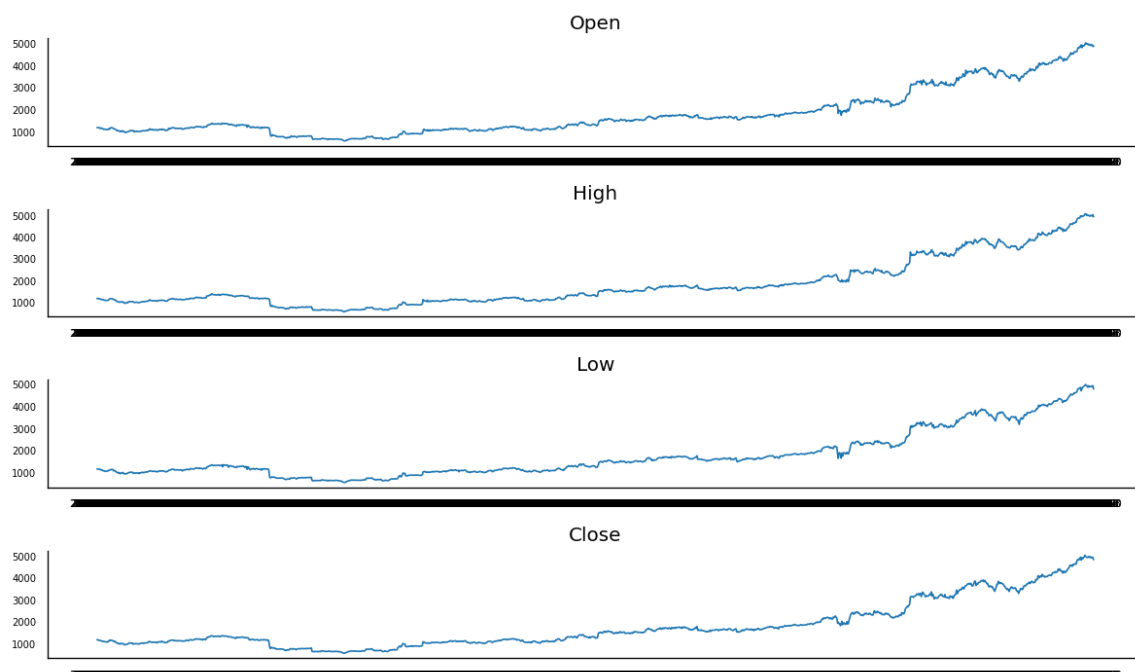
(1383, 4)

Plotting the Data

In [39]:

```
fig, axes = plt.subplots(nrows=4, ncols=1, dpi=120, figsize=(10,6))
for i, ax in enumerate(axes.flatten()):
    data = df_stock_var[df_stock_var.columns[i]]
    ax.plot(data, linewidth=1)
    # Decorations
    ax.set_title(df_stock_var.columns[i])
    ax.xaxis.set_ticks_position('none')
    ax.yaxis.set_ticks_position('none')
    ax.spines["top"].set_alpha(0)
    ax.tick_params(labelsize=6)

plt.tight_layout();
```



Granger Causality Test

In [40]:

```

from statsmodels.tsa.stattools import grangercausalitytests
maxlag=12
test = 'ssr_chi2test'
def grangers_causation_matrix(data, variables, test='ssr_chi2test', verbose=False):
    df = pd.DataFrame(np.zeros((len(variables), len(variables))), columns=variables, index=variables)
    for c in df.columns:
        for r in df.index:
            test_result = grangercausalitytests(data[[r, c]], maxlag=maxlag, verbose=False)
            p_values = [round(test_result[i+1][0][test][1],4) for i in range(maxlag)]
            if verbose: print(f'Y = {r}, X = {c}, P Values = {p_values}')
            min_p_value = np.min(p_values)
            df.loc[r, c] = min_p_value
    df.columns = [var + '_x' for var in variables]
    df.index = [var + '_y' for var in variables]
    return df

grangers_causation_matrix(df_stock_var, variables = df_stock_var.columns)

```

Out[40]:

	Open_x	High_x	Low_x	Close_x
Open_y	1.0000	0.0000	0.0000	0.0
High_y	0.0000	1.0000	0.0000	0.0
Low_y	0.0000	0.0000	1.0000	0.0
Close_y	0.0089	0.0016	0.1758	1.0

Cointegration Test

In [41]:

```

from statsmodels.tsa.vector_ar.vecm import coint_johansen

def cointegration_test(df_stock_var, alpha=0.05):
    out = coint_johansen(df_stock_var, -1, 5)
    d = {'0.90':0, '0.95':1, '0.99':2}
    traces = out.lr1
    cvts = out.cvt[:, d[str(1-alpha)]]
    def adjust(val, length= 6): return str(val).ljust(length)

    print('Name    :: Test Statistic > C(95%)    => Significance \n', '--'*20)
    for col, trace, cvt in zip(df_stock_var.columns, traces, cvts):
        print(adjust(col), ':: ', adjust(round(trace,2), 9), ">", adjust(cvt, 8), ' =>
', trace > cvt)

cointegration_test(df_stock_var)

```

Name	::	Test Statistic > C(95%)	=>	Significance
Open	::	478.92 > 40.1749	=>	True
High	::	267.15 > 24.2761	=>	True
Low	::	85.31 > 12.3212	=>	True
Close	::	9.69 > 4.1296	=>	True

ADF Test

In [42]:

```

def adfuller_test(series, signif=0.05, name='', verbose=False):
    """Perform ADFuller to test for Stationarity of given series and print report"""
    r = adfuller(series, autolag='AIC')
    output = {'test_statistic':round(r[0], 4), 'pvalue':round(r[1], 4), 'n_lags':round(
r[2], 4), 'n_obs':r[3]}
    p_value = output['pvalue']
    def adjust(val, length= 6): return str(val).ljust(length)

    # Print Summary
    print(f'    Augmented Dickey-Fuller Test on "{name}"', "\n    ", '-'*47)
    print(f' Null Hypothesis: Data has unit root. Non-Stationary.')
    print(f' Significance Level      = {signif}')
    print(f' Test Statistic            = {output["test_statistic"]}')
    print(f' No. Lags Chosen           = {output["n_lags"]}')

    for key, val in r[4].items():
        print(f' Critical value {adjust(key)} = {round(val, 3)}')

    if p_value <= signif:
        print(f" => P-Value = {p_value}. Rejecting Null Hypothesis.")
        print(f" => Series is Stationary.")
    else:
        print(f" => P-Value = {p_value}. Weak evidence to reject the Null Hypothesis.")
        print(f" => Series is Non-Stationary.")

```

In [43]:

```
# ADF Test on each column
for name, column in df_stock_var_train.iteritems():
    adfuller_test(column, name=column.name)
    print('\n')
```

Augmented Dickey-Fuller Test on "Open"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = 2.5158
No. Lags Chosen         = 10
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 0.9991. Weak evidence to reject the Null Hypothesis.
=> Series is Non-Stationary.
```

Augmented Dickey-Fuller Test on "High"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = 2.6903
No. Lags Chosen         = 1
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 0.9991. Weak evidence to reject the Null Hypothesis.
=> Series is Non-Stationary.
```

Augmented Dickey-Fuller Test on "Low"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = 2.4137
No. Lags Chosen         = 6
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 0.999. Weak evidence to reject the Null Hypothesis.
=> Series is Non-Stationary.
```

Augmented Dickey-Fuller Test on "Close"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = 2.9124
No. Lags Chosen         = 1
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 1.0. Weak evidence to reject the Null Hypothesis.
=> Series is Non-Stationary.
```

ADF Test on first order differencing

In [44]:

```
df_differenced = df_stock_var_train.diff().dropna()
```


In [45]:

```
# ADF Test on each column of 1st Differences Dataframe
for name, column in df_differenced.iteritems():
    adfuller_test(column, name=column.name)
    print('\n')
```

Augmented Dickey-Fuller Test on "Open"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = -11.3271
No. Lags Chosen         = 9
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 0.0. Rejecting Null Hypothesis.
=> Series is Stationary.
```

Augmented Dickey-Fuller Test on "High"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = -34.9968
No. Lags Chosen         = 0
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 0.0. Rejecting Null Hypothesis.
=> Series is Stationary.
```

Augmented Dickey-Fuller Test on "Low"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = -14.0262
No. Lags Chosen         = 5
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 0.0. Rejecting Null Hypothesis.
=> Series is Stationary.
```

Augmented Dickey-Fuller Test on "Close"

```
-----
Null Hypothesis: Data has unit root. Non-Stationary.
Significance Level      = 0.05
Test Statistic          = -38.8674
No. Lags Chosen         = 0
Critical value 1%       = -3.435
Critical value 5%       = -2.864
Critical value 10%      = -2.568
=> P-Value = 0.0. Rejecting Null Hypothesis.
=> Series is Stationary.
```

Selecting the number of laas

In [46]:

```
model = VAR(df_differenced)
x = model.select_order(maxlags=20)
x.summary()
```

Out[46]:

VAR Order Selection (* highlights the minimums)

	AIC	BIC	FPE	HQIC
0	26.99	27.00	5.244e+11	26.99
1	25.35	25.43	1.020e+11	25.38
2	24.88	25.02	6.363e+10	24.93
3	24.58	24.79	4.754e+10	24.66
4	24.47	24.73	4.235e+10	24.57
5	24.34	24.67*	3.730e+10	24.46
6	24.31	24.70	3.624e+10	24.46
7	24.29	24.74	3.532e+10	24.46
8	24.27	24.78	3.455e+10	24.46
9	24.22	24.80	3.313e+10	24.44*
10	24.20	24.84	3.249e+10	24.44
11	24.19	24.89	3.205e+10	24.45
12	24.19	24.94	3.196e+10	24.47
13	24.17	24.99	3.145e+10	24.48
14	24.16	25.04	3.113e+10	24.49
15	24.17	25.11	3.131e+10	24.52
16	24.17	25.18	3.153e+10	24.55
17	24.15	25.22	3.088e+10	24.55
18	24.15	25.28	3.075e+10	24.57
19	24.14*	25.33	3.048e+10*	24.59
20	24.15	25.40	3.073e+10	24.62

Fitting the model

In [47]:

```
model_fitted = model.fit(19)
model_fitted.summary()
```

Out[47]:

```

Summary of Regression Results
=====
Model:                VAR
Method:               OLS
Date:                Tue, 14, Sep, 2021
Time:                14:15:44
-----
No. of Equations:    4.00000    BIC:                25.3257
Nobs:                1349.00    HQIC:              24.5821
Log likelihood:      -23628.9    FPE:               3.03879e+10
AIC:                 24.1368    Det(Omega_mle):    2.43372e+10
-----
Results for equation Open
=====
==

```

	coefficient	std. error	t-stat	pr
const	0.662924	0.497410	1.333	0.1
L1.Open	-1.058015	0.039632	-26.696	0.0
L1.High	0.087818	0.033983	2.584	0.0
L1.Low	0.074787	0.030159	2.480	0.0
L1.Close	0.952053	0.033274	28.613	0.0
L2.Open	-1.017980	0.054720	-18.604	0.0
L2.High	0.110181	0.046567	2.366	0.0
L2.Low	0.117552	0.039498	2.976	0.0
L2.Close	0.869109	0.053643	16.202	0.0
L3.Open	-0.909310	0.066509	-13.672	0.0
L3.High	0.113335	0.056059	2.022	0.0
L3.Low	0.048514	0.048199	1.007	0.3
L3.Close	0.824324	0.068663	12.005	0.0
L4.Open	-0.746088	0.075751	-9.849	0.0
L4.High	0.074159	0.063291	1.172	0.2
L4.Low	0.031442	0.054952	0.572	0.5
L4.Close	0.734936	0.080234	9.160	0.0
L5.Open	-0.750765	0.081979	-9.158	0.0
L5.High	0.159277	0.068548	2.324	0.0
L5.Low	0.076147	0.059144	1.287	0.1

L5.Close 00	0.575474	0.087918	6.546	0.0
L6.Open 00	-0.720225	0.087255	-8.254	0.0
L6.High 12	0.183235	0.072847	2.515	0.0
L6.Low 30	0.136352	0.063022	2.164	0.0
L6.Close 00	0.449166	0.093315	4.813	0.0
L7.Open 00	-0.730536	0.090492	-8.073	0.0
L7.High 10	0.194867	0.075278	2.589	0.0
L7.Low 11	0.166413	0.065124	2.555	0.0
L7.Close 00	0.387159	0.096298	4.020	0.0
L8.Open 00	-0.634289	0.093372	-6.793	0.0
L8.High 98	0.127476	0.076951	1.657	0.0
L8.Low 09	0.173754	0.066625	2.608	0.0
L8.Close 00	0.359667	0.097832	3.676	0.0
L9.Open 00	-0.598336	0.095285	-6.279	0.0
L9.High 02	0.237665	0.077766	3.056	0.0
L9.Low 00	0.252208	0.067522	3.735	0.0
L9.Close 30	0.215217	0.098901	2.176	0.0
L10.Open 00	-0.512984	0.095881	-5.350	0.0
L10.High 01	0.268253	0.078086	3.435	0.0
L10.Low 01	0.232544	0.068135	3.413	0.0
L10.Close 75	0.070981	0.099366	0.714	0.4
L11.Open 00	-0.472701	0.094682	-4.993	0.0
L11.High 00	0.330399	0.078415	4.213	0.0
L11.Low 00	0.260047	0.067929	3.828	0.0
L11.Close 25	-0.097439	0.099102	-0.983	0.3
L12.Open 00	-0.488084	0.092693	-5.266	0.0
L12.High 00	0.387154	0.078070	4.959	0.0
L12.Low 01	0.228957	0.067179	3.408	0.0
L12.Close 72	-0.134179	0.098132	-1.367	0.1
L13.Open 00	-0.435278	0.089673	-4.854	0.0
L13.High	0.367114	0.077226	4.754	0.0

00				
L13.Low	0.194388	0.065881	2.951	0.0
03				
L13.Close	-0.094477	0.096575	-0.978	0.3
28				
L14.Open	-0.359888	0.085832	-4.193	0.0
00				
L14.High	0.242962	0.075571	3.215	0.0
01				
L14.Low	0.193441	0.063755	3.034	0.0
02				
L14.Close	-0.063773	0.093632	-0.681	0.4
96				
L15.Open	-0.232211	0.079937	-2.905	0.0
04				
L15.High	0.180999	0.071156	2.544	0.0
11				
L15.Low	0.059183	0.060025	0.986	0.3
24				
L15.Close	-0.007446	0.089058	-0.084	0.9
33				
L16.Open	-0.214780	0.072957	-2.944	0.0
03				
L16.High	0.171776	0.065563	2.620	0.0
09				
L16.Low	0.043893	0.055642	0.789	0.4
30				
L16.Close	0.012156	0.082745	0.147	0.8
83				
L17.Open	-0.079293	0.063304	-1.253	0.2
10				
L17.High	0.102741	0.057733	1.780	0.0
75				
L17.Low	-0.023318	0.049000	-0.476	0.6
34				
L17.Close	0.066324	0.072382	0.916	0.3
60				
L18.Open	-0.073340	0.049549	-1.480	0.1
39				
L18.High	0.040480	0.047515	0.852	0.3
94				
L18.Low	0.016085	0.040456	0.398	0.6
91				
L18.Close	0.027412	0.058524	0.468	0.6
40				
L19.Open	-0.008402	0.028976	-0.290	0.7
72				
L19.High	-0.012720	0.034276	-0.371	0.7
11				
L19.Low	-0.040443	0.030963	-1.306	0.1
92				
L19.Close	0.033426	0.041779	0.800	0.4
24				

=====

==

Results for equation High

=====

==

	coefficient	std. error	t-stat	pr
--	-------------	------------	--------	----

ob

--				
const	1.823387	0.838416	2.175	0.0
30				
L1.Open	-0.123752	0.066802	-1.853	0.0
64				
L1.High	-0.763053	0.057280	-13.321	0.0
00				
L1.Low	-0.048055	0.050834	-0.945	0.3
44				
L1.Close	0.919955	0.056085	16.403	0.0
00				
L2.Open	-0.005837	0.092233	-0.063	0.9
50				
L2.High	-0.667774	0.078492	-8.508	0.0
00				
L2.Low	-0.047040	0.066576	-0.707	0.4
80				
L2.Close	0.908117	0.090418	10.044	0.0
00				
L3.Open	0.146066	0.112105	1.303	0.1
93				
L3.High	-0.594883	0.094492	-6.296	0.0
00				
L3.Low	-0.129722	0.081243	-1.597	0.1
10				
L3.Close	0.711164	0.115735	6.145	0.0
00				
L4.Open	0.313078	0.127682	2.452	0.0
14				
L4.High	-0.659791	0.106680	-6.185	0.0
00				
L4.Low	-0.251031	0.092625	-2.710	0.0
07				
L4.Close	0.661824	0.135240	4.894	0.0
00				
L5.Open	0.248336	0.138181	1.797	0.0
72				
L5.High	-0.505316	0.115543	-4.373	0.0
00				
L5.Low	-0.230605	0.099690	-2.313	0.0
21				
L5.Close	0.484597	0.148192	3.270	0.0
01				
L6.Open	0.105920	0.147074	0.720	0.4
71				
L6.High	-0.382401	0.122788	-3.114	0.0
02				
L6.Low	-0.098558	0.106228	-0.928	0.3
54				
L6.Close	0.353870	0.157289	2.250	0.0
24				
L7.Open	0.124006	0.152530	0.813	0.4
16				
L7.High	-0.354310	0.126886	-2.792	0.0
05				
L7.Low	-0.122111	0.109771	-1.112	0.2
66				
L7.Close	0.381344	0.162316	2.349	0.0
19				
L8.Open	0.318732	0.157384	2.025	0.0

43				
L8.High	-0.376041	0.129705	-2.899	0.0
04				
L8.Low	-0.105205	0.112300	-0.937	0.3
49				
L8.Close	0.332100	0.164902	2.014	0.0
44				
L9.Open	0.197068	0.160608	1.227	0.2
20				
L9.High	-0.285367	0.131080	-2.177	0.0
29				
L9.Low	-0.018559	0.113812	-0.163	0.8
70				
L9.Close	0.058274	0.166704	0.350	0.7
27				
L10.Open	0.189402	0.161614	1.172	0.2
41				
L10.High	-0.223236	0.131618	-1.696	0.0
90				
L10.Low	-0.003104	0.114846	-0.027	0.9
78				
L10.Close	0.060209	0.167488	0.359	0.7
19				
L11.Open	0.100924	0.159592	0.632	0.5
27				
L11.High	-0.157505	0.132174	-1.192	0.2
33				
L11.Low	-0.007869	0.114499	-0.069	0.9
45				
L11.Close	-0.037764	0.167042	-0.226	0.8
21				
L12.Open	0.051419	0.156240	0.329	0.7
42				
L12.High	-0.054530	0.131593	-0.414	0.6
79				
L12.Low	-0.089062	0.113235	-0.787	0.4
32				
L12.Close	0.103503	0.165409	0.626	0.5
31				
L13.Open	0.023515	0.151150	0.156	0.8
76				
L13.High	0.028854	0.130169	0.222	0.8
25				
L13.Low	-0.055504	0.111046	-0.500	0.6
17				
L13.Close	0.032383	0.162783	0.199	0.8
42				
L14.Open	0.009691	0.144675	0.067	0.9
47				
L14.High	0.011484	0.127379	0.090	0.9
28				
L14.Low	-0.010881	0.107463	-0.101	0.9
19				
L14.Close	-0.038428	0.157822	-0.243	0.8
08				
L15.Open	0.005029	0.134739	0.037	0.9
70				
L15.High	-0.020062	0.119939	-0.167	0.8
67				
L15.Low	-0.057325	0.101176	-0.567	0.5
71				

L15.Close 51	0.009142	0.150112	0.061	0.9
L16.Open 57	0.091533	0.122974	0.744	0.4
L16.High 66	0.047732	0.110510	0.432	0.6
L16.Low 39	-0.138786	0.093788	-1.480	0.1
L16.Close 50	0.130378	0.139472	0.935	0.3
L17.Open 63	0.198241	0.106703	1.858	0.0
L17.High 02	-0.081470	0.097313	-0.837	0.4
L17.Low 05	-0.231958	0.082593	-2.808	0.0
L17.Close 00	0.102577	0.122005	0.841	0.4
L18.Open 63	0.155376	0.083518	1.860	0.0
L18.High 29	-0.096342	0.080089	-1.203	0.2
L18.Low 56	-0.130474	0.068192	-1.913	0.0
L18.Close 36	0.033298	0.098647	0.338	0.7
L19.Open 34	0.103467	0.048841	2.118	0.0
L19.High 54	-0.111170	0.057774	-1.924	0.0
L19.Low 05	-0.144987	0.052191	-2.778	0.0
L19.Close 52	0.100994	0.070421	1.434	0.1

=====

==

Results for equation Low

=====

==

	coefficient	std. error	t-stat	pr
ob				
--				
const 02	1.469116	0.898507	1.635	0.1
L1.Open 17	-0.171428	0.071590	-2.395	0.0
L1.High 20	-0.049548	0.061385	-0.807	0.4
L1.Low 00	-0.711393	0.054478	-13.058	0.0
L1.Close 00	0.909481	0.060105	15.132	0.0
L2.Open 58	-0.057882	0.098844	-0.586	0.5
L2.High 88	-0.089322	0.084118	-1.062	0.2
L2.Low 00	-0.699476	0.071347	-9.804	0.0
L2.Close	0.959028	0.096899	9.897	0.0

00				
L3.Open	0.040094	0.120140	0.334	0.7
39				
L3.High	-0.104054	0.101264	-1.028	0.3
04				
L3.Low	-0.659928	0.087065	-7.580	0.0
00				
L3.Close	0.834346	0.124030	6.727	0.0
00				
L4.Open	0.179307	0.136834	1.310	0.1
90				
L4.High	-0.237302	0.114326	-2.076	0.0
38				
L4.Low	-0.635733	0.099264	-6.404	0.0
00				
L4.Close	0.738241	0.144933	5.094	0.0
00				
L5.Open	0.122364	0.148085	0.826	0.4
09				
L5.High	-0.144022	0.123824	-1.163	0.2
45				
L5.Low	-0.585459	0.106835	-5.480	0.0
00				
L5.Close	0.617993	0.158813	3.891	0.0
00				
L6.Open	0.077634	0.157615	0.493	0.6
22				
L6.High	-0.157320	0.131589	-1.196	0.2
32				
L6.Low	-0.426953	0.113842	-3.750	0.0
00				
L6.Close	0.528931	0.168562	3.138	0.0
02				
L7.Open	0.084304	0.163462	0.516	0.6
06				
L7.High	-0.086508	0.135980	-0.636	0.5
25				
L7.Low	-0.407936	0.117638	-3.468	0.0
01				
L7.Close	0.496511	0.173949	2.854	0.0
04				
L8.Open	0.166173	0.168664	0.985	0.3
25				
L8.High	-0.024993	0.139001	-0.180	0.8
57				
L8.Low	-0.399196	0.120349	-3.317	0.0
01				
L8.Close	0.321954	0.176721	1.822	0.0
68				
L9.Open	0.038142	0.172119	0.222	0.8
25				
L9.High	0.109781	0.140474	0.781	0.4
35				
L9.Low	-0.253917	0.121969	-2.082	0.0
37				
L9.Close	0.112740	0.178652	0.631	0.5
28				
L10.Open	-0.026030	0.173197	-0.150	0.8
81				
L10.High	0.265385	0.141052	1.881	0.0
60				

L10.Low 23	-0.121567	0.123077	-0.988	0.3
L10.Close 47	-0.082193	0.179492	-0.458	0.6
L11.Open 70	-0.097256	0.171031	-0.569	0.5
L11.High 09	0.369588	0.141647	2.609	0.0
L11.Low 97	-0.015813	0.122706	-0.129	0.8
L11.Close 16	-0.281584	0.179015	-1.573	0.1
L12.Open 98	-0.141494	0.167438	-0.845	0.3
L12.High 03	0.414344	0.141024	2.938	0.0
L12.Low 89	-0.065643	0.121350	-0.541	0.5
L12.Close 79	-0.238286	0.177264	-1.344	0.1
L13.Open 93	-0.170228	0.161983	-1.051	0.2
L13.High 02	0.433163	0.139499	3.105	0.0
L13.Low 63	-0.005474	0.119005	-0.046	0.9
L13.Close 71	-0.239058	0.174450	-1.370	0.1
L14.Open 67	-0.088703	0.155044	-0.572	0.5
L14.High 65	0.252283	0.136508	1.848	0.0
L14.Low 84	0.046835	0.115165	0.407	0.6
L14.Close 81	-0.182437	0.169133	-1.079	0.2
L15.Open 42	-0.067166	0.144396	-0.465	0.6
L15.High 80	0.112798	0.128535	0.878	0.3
L15.Low 01	-0.027309	0.108428	-0.252	0.8
L15.Close 80	-0.088981	0.160871	-0.553	0.5
L16.Open 21	-0.161154	0.131787	-1.223	0.2
L16.High 63	0.220561	0.118430	1.862	0.0
L16.Low 53	0.005911	0.100510	0.059	0.9
L16.Close 72	-0.063268	0.149469	-0.423	0.6
L17.Open 89	-0.045727	0.114350	-0.400	0.6
L17.High 78	0.091994	0.104287	0.882	0.3
L17.Low 70	-0.063900	0.088512	-0.722	0.4
L17.Close 56	0.058256	0.130749	0.446	0.6
L18.Open	-0.171490	0.089503	-1.916	0.0

55				
L18.High	0.133388	0.085830	1.554	0.1
20				
L18.Low	0.128271	0.073079	1.755	0.0
79				
L18.Close	-0.094805	0.105717	-0.897	0.3
70				
L19.Open	-0.060919	0.052341	-1.164	0.2
44				
L19.High	0.052802	0.061915	0.853	0.3
94				
L19.Low	-0.013859	0.055931	-0.248	0.8
04				
L19.Close	-0.000534	0.075468	-0.007	0.9
94				

=====

==

Results for equation Close

=====

==

	coefficient	std. error	t-stat	pr
ob				
--				
const	2.431974	1.071989	2.269	0.0
23				
L1.Open	-0.048618	0.085412	-0.569	0.5
69				
L1.High	0.125734	0.073237	1.717	0.0
86				
L1.Low	0.061215	0.064996	0.942	0.3
46				
L1.Close	-0.158455	0.071710	-2.210	0.0
27				
L2.Open	0.107598	0.117928	0.912	0.3
62				
L2.High	0.167088	0.100359	1.665	0.0
96				
L2.Low	0.110794	0.085123	1.302	0.1
93				
L2.Close	-0.180164	0.115608	-1.558	0.1
19				
L3.Open	0.273389	0.143337	1.907	0.0
56				
L3.High	0.183006	0.120816	1.515	0.1
30				
L3.Low	0.092107	0.103876	0.887	0.3
75				
L3.Close	-0.389521	0.147978	-2.632	0.0
08				
L4.Open	0.463774	0.163253	2.841	0.0
04				
L4.High	-0.044975	0.136400	-0.330	0.7
42				
L4.Low	-0.105126	0.118429	-0.888	0.3
75				
L4.Close	-0.321029	0.172916	-1.857	0.0
63				
L5.Open	0.379195	0.176677	2.146	0.0
32				

L5.High 94	0.058039	0.147732	0.393	0.6
L5.Low 93	-0.087330	0.127463	-0.685	0.4
L5.Close 40	-0.389860	0.189476	-2.058	0.0
L6.Open 12	0.298747	0.188047	1.589	0.1
L6.High 76	0.004650	0.156996	0.030	0.9
L6.Low 64	-0.040700	0.135822	-0.300	0.7
L6.Close 24	-0.309658	0.201108	-1.540	0.1
L7.Open 54	0.376524	0.195023	1.931	0.0
L7.High 53	0.009559	0.162235	0.059	0.9
L7.Low 10	-0.115655	0.140352	-0.824	0.4
L7.Close 75	-0.184017	0.207535	-0.887	0.3
L8.Open 02	0.614609	0.201229	3.054	0.0
L8.High 62	-0.007990	0.165839	-0.048	0.9
L8.Low 71	-0.157940	0.143586	-1.100	0.2
L8.Close 89	-0.276832	0.210841	-1.313	0.1
L9.Open 17	0.488579	0.205352	2.379	0.0
L9.High 29	-0.036286	0.167597	-0.217	0.8
L9.Low 33	-0.090625	0.145519	-0.623	0.5
L9.Close 26	-0.474053	0.213146	-2.224	0.0
L10.Open 66	0.380186	0.206637	1.840	0.0
L10.High 33	0.080338	0.168286	0.477	0.6
L10.Low 12	0.034917	0.146841	0.238	0.8
L10.Close 17	-0.510500	0.214148	-2.384	0.0
L11.Open 44	0.298163	0.204053	1.461	0.1
L11.High 43	0.102840	0.168996	0.609	0.5
L11.Low 12	0.074350	0.146397	0.508	0.6
L11.Close 10	-0.549972	0.213579	-2.575	0.0
L12.Open 02	0.206281	0.199767	1.033	0.3
L12.High 85	0.146284	0.168253	0.869	0.3
L12.Low 56	-0.044990	0.144780	-0.311	0.7
L12.Close	-0.368168	0.211489	-1.741	0.0

82				
L13.Open	0.151299	0.193259	0.783	0.4
34				
L13.High	0.229431	0.166433	1.379	0.1
68				
L13.Low	0.025523	0.141982	0.180	0.8
57				
L13.Close	-0.357956	0.208132	-1.720	0.0
85				
L14.Open	0.167295	0.184980	0.904	0.3
66				
L14.High	0.127452	0.162865	0.783	0.4
34				
L14.Low	0.056802	0.137401	0.413	0.6
79				
L14.Close	-0.379570	0.201789	-1.881	0.0
60				
L15.Open	0.136529	0.172276	0.793	0.4
28				
L15.High	0.053843	0.153352	0.351	0.7
26				
L15.Low	0.021313	0.129363	0.165	0.8
69				
L15.Close	-0.291750	0.191932	-1.520	0.1
28				
L16.Open	0.137740	0.157233	0.876	0.3
81				
L16.High	0.169345	0.141297	1.199	0.2
31				
L16.Low	-0.059572	0.119916	-0.497	0.6
19				
L16.Close	-0.160563	0.178328	-0.900	0.3
68				
L17.Open	0.288274	0.136429	2.113	0.0
35				
L17.High	0.004685	0.124423	0.038	0.9
70				
L17.Low	-0.194550	0.105602	-1.842	0.0
65				
L17.Close	-0.059992	0.155994	-0.385	0.7
01				
L18.Open	0.110974	0.106785	1.039	0.2
99				
L18.High	0.044508	0.102401	0.435	0.6
64				
L18.Low	0.030696	0.087189	0.352	0.7
25				
L18.Close	-0.228281	0.126129	-1.810	0.0
70				
L19.Open	0.042960	0.062447	0.688	0.4
91				
L19.High	-0.052895	0.073869	-0.716	0.4
74				
L19.Low	-0.079500	0.066731	-1.191	0.2
34				
L19.Close	-0.049364	0.090040	-0.548	0.5
84				

=====

==

Correlation matrix of residuals

	Open	High	Low	Close
Open	1.000000	0.543935	0.531180	0.383960
High	0.543935	1.000000	0.572630	0.805821
Low	0.531180	0.572630	1.000000	0.788302
Close	0.383960	0.805821	0.788302	1.000000

Durbin Watson Test

In [48]:

```
from statsmodels.stats.stattools import durbin_watson
out = durbin_watson(model_fitted.resid)

for col, val in zip(df_stock_var.columns, out):
    print(col, ': ', round(val, 2))
```

Open : 1.99
High : 2.0
Low : 1.99
Close : 2.0

Getting the Forecast

In [49]:

```
# Get the Lag order
lag_order = model_fitted.k_ar
print(lag_order)

# Input data for forecasting
forecast_input = df_differenced.values[-lag_order:]
forecast_input
```

19

Out[49]:

```
array([[ 98.29980469,  52.94970703,  86.25      ,  40.30029297],
       [ 31.        , -1.04980469, -5.14990234, -35.75      ],
       [-65.        , -15.95019531, -27.10009766,  28.34960938],
       [ 33.        , -6.04980469,  15.        , -42.04980469],
       [-18.        ,  65.        ,  10.79980469,  89.20019531],
       [ 95.        , -1.04980469,  64.85009766, -10.95019531],
       [-10.        , -5.20019531,  -8.75      ,  17.29980469],
       [  9.5       ,  42.89990234,  18.25      ,  17.15039062],
       [ 16.5       ,  -9.25      ,  12.75      , -15.90039062],
       [ -1.        , 119.60009766,  22.10009766, 150.30029297],
       [124.95019531,  60.70019531, 115.54980469,  49.85009766],
       [ 57.04980469,  15.29980469,  27.45019531, -19.60009766],
       [-19.14990234,   8.39990234,   6.39990234,  55.39990234],
       [ 55.60009766,  -6.39990234,  32.30029297, -18.5       ],
       [-18.70019531,  90.64990234,  23.04980469,  96.39990234],
       [105.25      ,   4.10009766, -155.35009766, -129.75      ],
       [-127.39990234, -30.79980469,  98.85009766,  99.20019531],
       [106.29980469,  20.44970703,  55.95019531, -46.80029297],
       [-28.94970703,   6.55029297,  38.04980469,  62.30029297]])
```

In [50]:

```
nobs = 14
fc = model_fitted.forecast(y=forecast_input, steps=nobs)
df_forecast = pd.DataFrame(fc, index=df_stock_var.index[-nobs:], columns=df_stock_var.columns + '_1d')
df_forecast
```

Out[50]:

	Open_1d	High_1d	Low_1d	Close_1d
Date				
2021-08-02	37.293671	30.063593	7.769488	21.363187
2021-08-03	1.914199	22.989322	2.455674	17.500867
2021-08-04	31.753401	12.923545	26.275951	21.753088
2021-08-05	6.451376	13.890168	4.829672	9.948701
2021-08-06	18.108927	12.716636	32.567128	14.595948
2021-08-09	1.416460	-6.284053	-19.676285	-20.184323
2021-08-10	-0.575837	7.351079	9.260740	13.482345
2021-08-11	16.601844	18.112808	9.052183	13.249416
2021-08-12	13.426323	-4.188215	19.118470	8.587273
2021-08-13	2.218920	17.553188	1.714692	7.506405
2021-08-16	-8.088607	-9.061545	-18.736722	-6.700707
2021-08-17	22.971695	12.913013	12.195768	2.263554
2021-08-18	-16.453352	5.205200	-4.553027	13.441649
2021-08-20	26.734759	16.835577	23.195475	18.778101

Inverting forecast back to original scale

In [51]:

```
def invert_transformation(df_train, df_forecast, first_diff=False):
    """Revert back the differencing to get the forecast to original scale."""
    df_fc = df_forecast.copy()
    columns = df_train.columns
    for col in columns:
        if first_diff:
            # Roll back 1st Diff
            df_fc[str(col)+'_forecast'] = df_train[col].iloc[-1] + df_fc[str(col)+'_1d'].
cumsum()
    return df_fc
```


In [52]:

```
df_results = invert_transformation(df_stock_var_train, df_forecast, first_diff=True)
df_results.loc[:, ['Open_forecast', 'High_forecast', 'Low_forecast', 'Close_forecast']]
```

Out[52]:

	Open_forecast	High_forecast	Low_forecast	Close_forecast
Date				
2021-08-02	4917.243866	4971.013788	4870.019488	4927.463285
2021-08-03	4919.158065	4994.003111	4872.475162	4944.964152
2021-08-04	4950.911467	5006.926655	4898.751114	4966.717240
2021-08-05	4957.362843	5020.816823	4903.580786	4976.665941
2021-08-06	4975.471770	5033.533460	4936.147914	4991.261888
2021-08-09	4976.888230	5027.249407	4916.471629	4971.077565
2021-08-10	4976.312393	5034.600486	4925.732369	4984.559910
2021-08-11	4992.914237	5052.713294	4934.784551	4997.809327
2021-08-12	5006.340561	5048.525079	4953.903022	5006.396599
2021-08-13	5008.559480	5066.078267	4955.617714	5013.903004
2021-08-16	5000.470873	5057.016721	4936.880991	5007.202297
2021-08-17	5023.442568	5069.929735	4949.076759	5009.465851
2021-08-18	5006.989215	5075.134935	4944.523733	5022.907500
2021-08-20	5033.723974	5091.970513	4967.719208	5041.685602

Plotting Forecast vs Actual

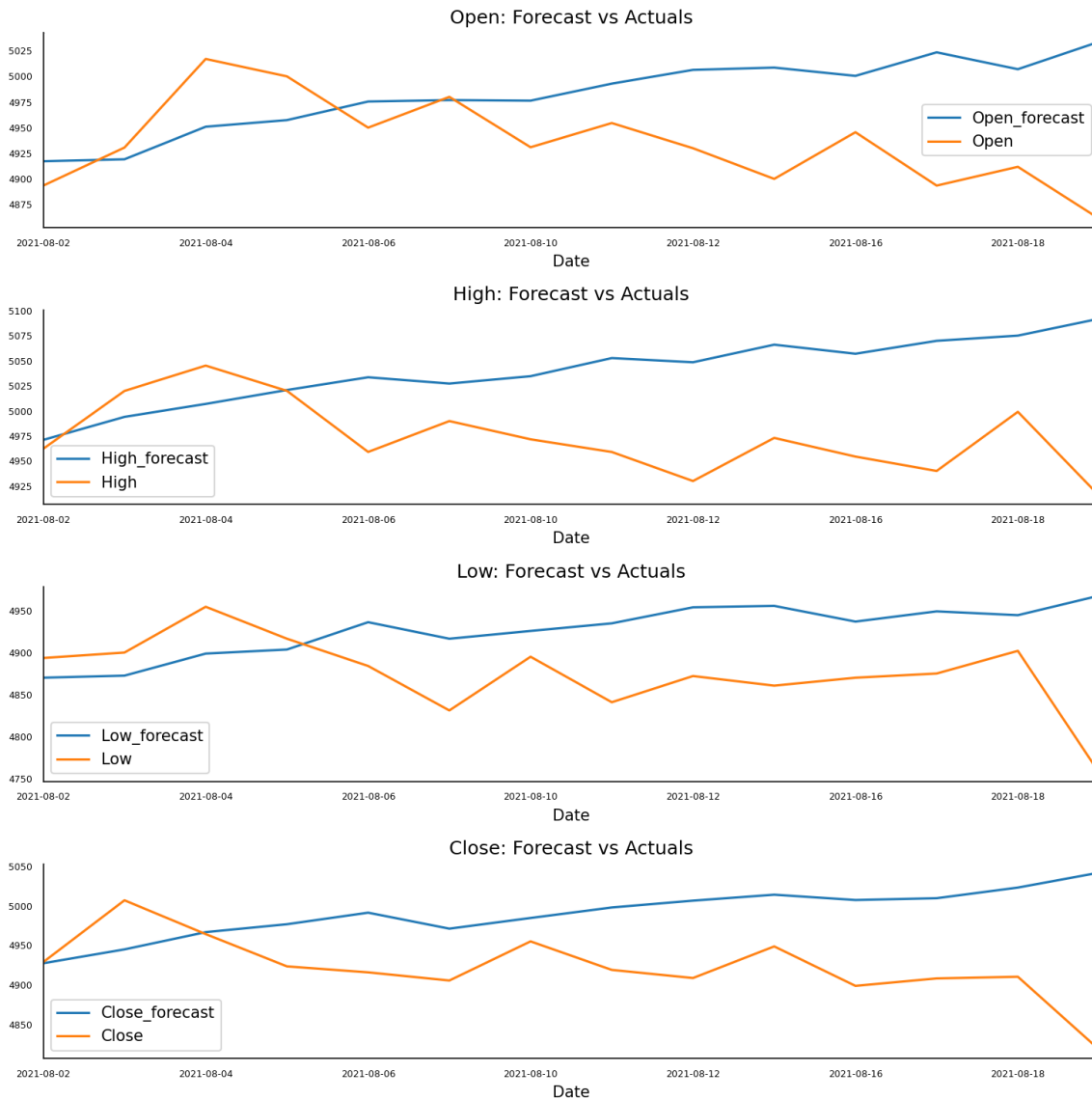
In [53]:

```

fig, axes = plt.subplots(nrows=4, ncols=1, dpi=150, figsize=(10,10))
for i, (col,ax) in enumerate(zip(df_stock_var.columns, axes.flatten())):
    df_results[col+'_forecast'].plot(legend=True, ax=ax).autoscale(axis='x',tight=True)
    df_stock_var_test[col][-nobs:].plot(legend=True, ax=ax);
    ax.set_title(col + ": Forecast vs Actuals")
    ax.xaxis.set_ticks_position('none')
    ax.yaxis.set_ticks_position('none')
    ax.spines["top"].set_alpha(0)
    ax.tick_params(labelsize=6)

plt.tight_layout();

```



Evaluating the model

In [54]:

```

from statsmodels.tsa.stattools import acf
def forecast_accuracy(forecast, actual):
    rmse = np.mean((forecast - actual)**2)**.5
    rmse_p = (rmse/np.mean(actual))*100
    return({'rmse':rmse,'rmse_mean':rmse_p})

print('Forecast Accuracy of Open')
accuracy_prod = forecast_accuracy(df_results['Open_forecast'].values, df_stock_var_test['Open'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))

print('\nForecast Accuracy of High')
accuracy_prod = forecast_accuracy(df_results['High_forecast'].values, df_stock_var_test['High'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))

print('\nForecast Accuracy of Low')
accuracy_prod = forecast_accuracy(df_results['Low_forecast'].values, df_stock_var_test['Low'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))

print('\nForecast Accuracy of Close')
accuracy_prod = forecast_accuracy(df_results['Close_forecast'].values, df_stock_var_test['Close'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))

```

Forecast Accuracy of Open

rmse : 78.998

rmse_mean : 1.6006

Forecast Accuracy of High

rmse : 88.2549

rmse_mean : 1.7743

Forecast Accuracy of Low

rmse : 82.9411

rmse_mean : 1.7013

Forecast Accuracy of Close

rmse : 93.2519

rmse_mean : 1.8944

Linear Regression

In [55]:

```

df_stock_train_lin=df_stock.loc['2016-01-01':'2021-07-31'].copy()
df_stock_test_lin=df_stock.loc['2021-07-31:'].copy()

```

Normalize the data

In [56]:

```
from sklearn.preprocessing import MinMaxScaler
```

In [57]:

```
scaler = MinMaxScaler()
df_stock_train_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']] = scaler.fit_transform(df_stock_train_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']])
df_stock_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']] = scaler.transform(df_stock_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']])
```

In [58]:

```
df_stock_train_lin.head()
```

Out[58]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	NIFTY50
Date									
2016-01-01	0.139079	0.138096	0.141560	1162.500000	0.000084	4	1	1	0.091661
2016-01-04	0.140675	0.140080	0.139770	1149.699951	0.006552	0	4	1	0.090919
2016-01-05	0.137483	0.136386	0.139932	1147.500000	0.009918	1	5	1	0.086044
2016-01-06	0.137939	0.135656	0.139124	1137.199951	0.014293	2	6	1	0.066755
2016-01-07	0.130187	0.131609	0.134504	1126.949951	0.010612	3	7	1	0.070447

In [59]:

```
df_stock_test_lin.head()
```

Out[59]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	NIFTY50
Date									
2021-08-02	0.991678	1.003968	1.007218	4928.950195	0.012690	0	2	8	1.023069
2021-08-03	1.000137	1.017137	1.008719	5006.850098	0.021029	1	3	8	1.037370
2021-08-04	1.019836	1.022906	1.021318	4964.250000	0.013759	2	4	8	1.041369
2021-08-05	1.015960	1.017159	1.012530	4923.549805	0.015311	3	5	8	1.035070
2021-08-06	1.004560	1.003249	1.005024	4916.000000	0.010502	4	6	8	1.037309

Checking for multicollinearity

In [60]:

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

def calculate_vif(X, thresh=10, verbose=True):
    cols = X.columns
    variables = np.arange(X.shape[1])
    dropped=True
    while dropped:
        dropped=False
        c = X[cols[variables]].values
        vif = [variance_inflation_factor(c, ix) for ix in np.arange(c.shape[1])]

        maxloc = vif.index(max(vif))
        if max(vif) > thresh:
            if verbose == True:
                print('dropping \'' + X[cols[variables]].columns[maxloc] + '\' at inde
x: ' + str(maxloc))
            variables = np.delete(variables, maxloc)
            dropped=True
    if verbose == True:
        print('Remaining variables:')
        print(X.columns[variables])
    return X[cols[variables]]
```

Model Generation

In [61]:

```
def forward_regression(X, y,
                      threshold_in,
                      verbose=False):
    initial_list = []
    included = list(initial_list)
    while True:
        changed=False
        excluded = list(set(X.columns)-set(included))
        new_pval = pd.Series(index=excluded)
        for new_column in excluded:
            model = sm.OLS(y, sm.add_constant(pd.DataFrame(X[included+[new_column]]))).
fit()
            new_pval[new_column] = model.pvalues[new_column]
            best_pval = new_pval.min()
            if best_pval < threshold_in:
                best_feature = new_pval.idxmin()
                included.append(best_feature)
                changed=True
                if verbose:
                    print('Add {:30} with p-value {:.6}'.format(best_feature, best_pval))

            if not changed:
                break

    return included
```

In [62]:

```
def backward_regression(X, y,
                       threshold_out,
                       verbose=False):
    included=list(X.columns)
    while True:
        changed=False
        model = sm.OLS(y, sm.add_constant(pd.DataFrame(X[included]))).fit()
        pvalues = model.pvalues.iloc[1:]
        worst_pval = pvalues.max()
        if worst_pval > threshold_out:
            changed=True
            worst_feature = pvalues.idxmax()
            included.remove(worst_feature)
            if verbose:
                print('Drop {:30} with p-value {:.6}'.format(worst_feature, worst_pval
))
            if not changed:
                break
    return included
```

In [63]:

```
def model_generator(df_train,df_test):
    train_dummy=pd.get_dummies(df_train,columns=['Day of Week','Day of Month','Month'])
    test_dummy = pd.get_dummies(df_test,columns=['Day of Week','Day of Month','Month'])
    for i in [1,2,3,4,5,6,7,9,10,11,12]:
        test_dummy['Month_'+str(i)] = 0
        test_dummy['Month_'+str(i)] = test_dummy['Month_'+str(i)].astype('uint8')
    for i in range(1,32):
        if (i not in df_test['Day of Month'].unique()):
            test_dummy['Day of Month_'+str(i)] = 0
            test_dummy['Day of Month_'+str(i)] = test_dummy['Day of Month_'+str(i)].astype(
'uint8')
    X=train_dummy.drop('Close',axis=1)
    y=train_dummy["Close"]
    X_f=calculate_vif_(X.select_dtypes(exclude='uint8'),10,False)
    X_f=pd.concat([X_f,X.iloc[:,6:]],axis=1)
    model=sm.OLS(y,X_f[backward_regression(X_f,y,0.01)],axis=1).fit()
    pred=model.predict(test_dummy[backward_regression(X_f,y,0.01)])
    rmse = math.sqrt(sklearn.metrics.mean_squared_error(test_dummy['Close'], pred))
    rmse_pct= (rmse/np.mean(test_dummy['Close']))*100
    sig_var = model.pvalues.index.tolist()
    return model,sig_var,rmse_pct
```

In [64]:

```
model,variables,rmse_per = model_generator(df_stock_train_lin,df_stock_test_lin)
```

In [65]:

```
print(model.summary2())
```

Results: Ordinary least squares

```
=====
Model:                OLS                Adj. R-squared:    0.999
Dependent Variable:    Close              AIC:              13662.6638
Date:                 2021-09-14 14:15    BIC:              13751.4350
No. Observations:     1369              Log-Likelihood:    -6814.3
Df Model:              16                F-statistic:       7.123e+04
Df Residuals:          1352             Prob (F-statistic): 0.00
R-squared:             0.999             Scale:            1248.6
=====
```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
Open	4382.9999	4.1262	1062.2245	0.0000	4374.9053	4391.0944
Day of Week_0	381.0991	2.1942	173.6846	0.0000	376.7947	385.4035
Day of Week_1	382.2708	2.1760	175.6796	0.0000	378.0021	386.5394
Day of Week_2	382.7929	2.1569	177.4747	0.0000	378.5617	387.0241
Day of Week_3	386.7002	2.1662	178.5114	0.0000	382.4506	390.9498
Day of Week_4	387.8028	2.1856	177.4373	0.0000	383.5154	392.0903
Month_1	153.7430	3.0111	51.0581	0.0000	147.8360	159.6500
Month_2	160.4697	3.1564	50.8395	0.0000	154.2778	166.6617
Month_3	160.3193	3.1042	51.6462	0.0000	154.2298	166.4088
Month_4	163.8607	3.2159	50.9525	0.0000	157.5519	170.1695
Month_5	156.7296	3.0438	51.4916	0.0000	150.7586	162.7007
Month_6	159.8033	3.0557	52.2959	0.0000	153.8087	165.7978
Month_7	167.0932	3.0319	55.1117	0.0000	161.1455	173.0410
Month_8	163.0556	3.3284	48.9887	0.0000	156.5261	169.5850
Month_9	158.9598	3.4005	46.7464	0.0000	152.2890	165.6305
Month_10	160.5114	3.4320	46.7686	0.0000	153.7787	167.2441
Month_11	159.2133	3.3869	47.0087	0.0000	152.5691	165.8574
Month_12	156.9070	3.3837	46.3721	0.0000	150.2692	163.5447

```
-----
Omnibus:              229.237            Durbin-Watson:      2.239
Prob(Omnibus):        0.000            Jarque-Bera (JB):   2268.361
Skew:                 -0.454            Prob(JB):           0.000
Kurtosis:              9.240            Condition No.:      6635984677100544
=====
```

* The condition number is large (7e+15). This might indicate strong multicollinearity or other numerical problems.

In [66]:

```
print("Significance Variables :", '\n', *variables, sep='\n')
```

Significance Variables :

Open
Day of Week_0
Day of Week_1
Day of Week_2
Day of Week_3
Day of Week_4
Month_1
Month_2
Month_3
Month_4
Month_5
Month_6
Month_7
Month_8
Month_9
Month_10
Month_11
Month_12

In [67]:

```
print("RMSE Percentage", rmse_per, sep='\n')
```

RMSE Percentage
0.8986093085074989

Univariate Forecasting with ARIMA for Linear Regression

In [68]:

```
print("Forecasting Variables :", '\n', list(set(variables) & set(list(df_stock.columns  
))), sep='\n')
```

Forecasting Variables :

['Open']

In [69]:

```
df_stock_for_test_lin=df_stock_test_lin.iloc[:14].copy()  
dummy_df = pd.DataFrame(index=df_stock_for_test_lin.index)
```

In [70]:

```

Stationary=False
dvalue=0
Stationary_df=df_stock['Open'].copy()
while(Stationary==False):
    dfctest = adfuller(Stationary_df.dropna(),autolag='AIC')
    dfoutput=pd.Series(dfctest[0:4],index=['Test Statistic','p-value','#lags used','number
of observations used'])
    for key,value in dfctest[4].items():
        dfoutput['critical value (%s)'%key]= value

    if (dfctest[1]>0.05):
        Stationary=False
        Stationary_df = Stationary_df - Stationary_df.shift(1)
    else :
        Stationary=True
    dvalue+=1
print("Differencing Value : ",dvalue)

```

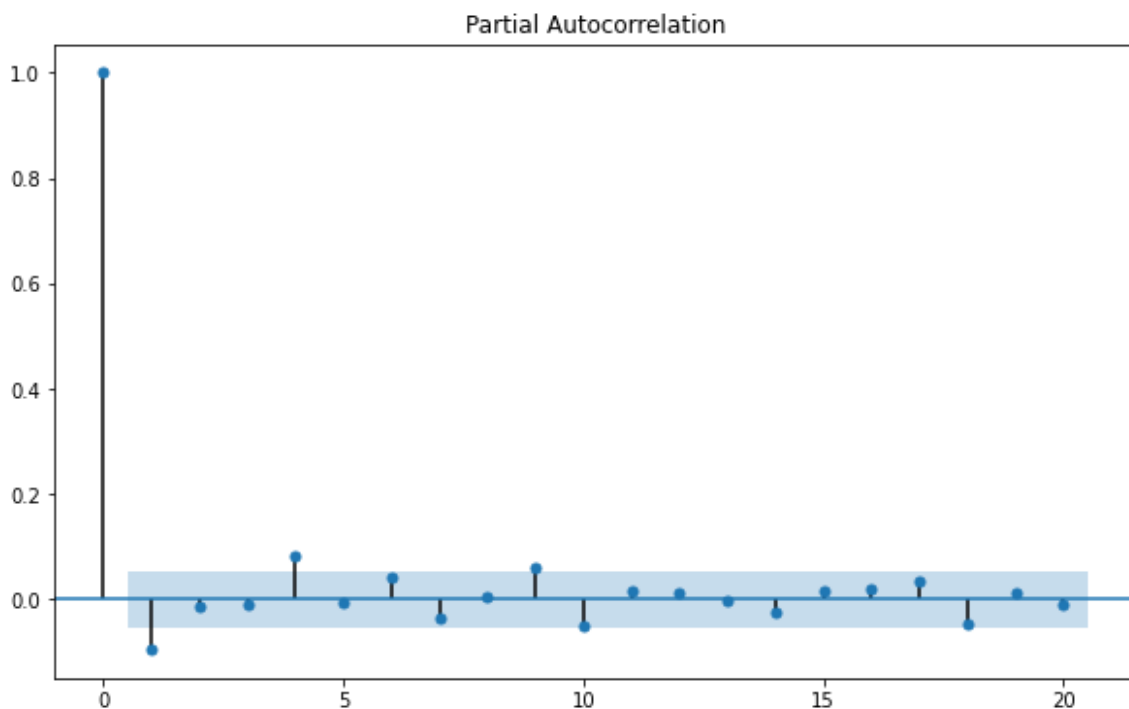
Differencing Value : 1

In [71]:

```

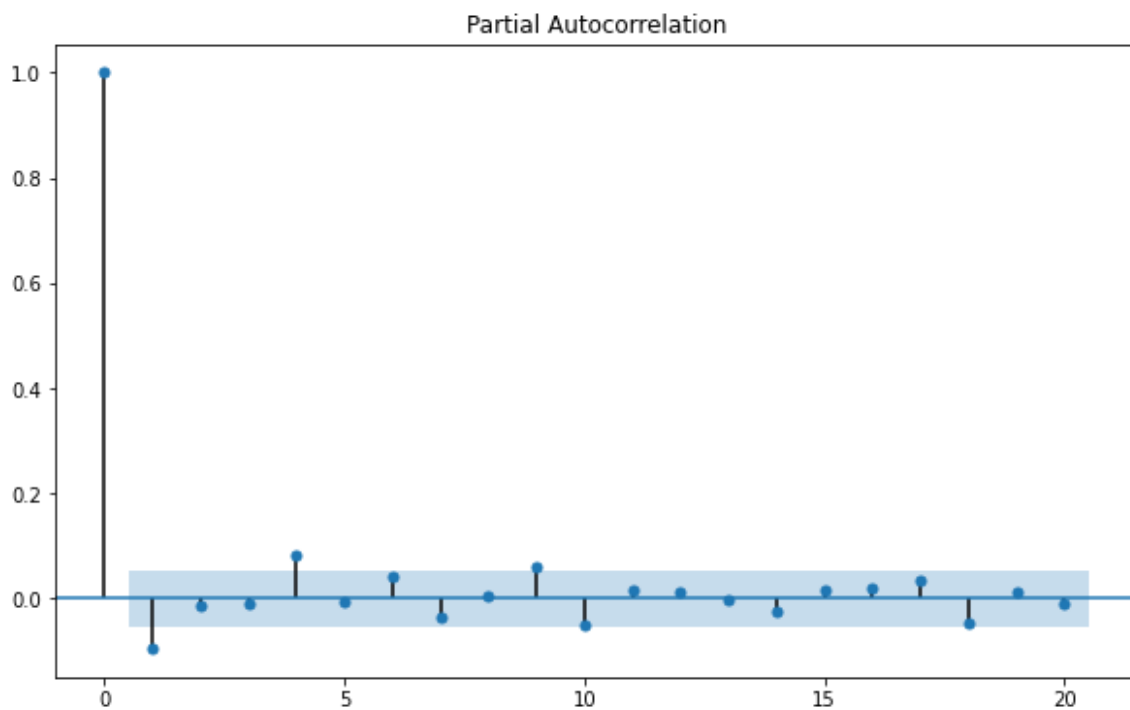
plt.pacf(Stationary_df.dropna(),lags=20).show()
plt.show()

```



In [72]:

```
plt.pacf(Stationary_df.dropna(),lags=20).show()  
plt.show()
```



In [73]:

```
df_stock_train=df_stock.loc['2016-01-01':'2021-07-31']['Open']  
df_stock_test=df_stock.loc['2021-07-31:']['Open']
```

In [74]:

```
df_stock_test
```

Out[74]:

```
Date
2021-08-02    4893.500000
2021-08-03    4930.600098
2021-08-04    5017.000000
2021-08-05    5000.000000
2021-08-06    4950.000000
2021-08-09    4980.000000
2021-08-10    4930.899902
2021-08-11    4954.500000
2021-08-12    4929.950195
2021-08-13    4900.049805
2021-08-16    4945.549805
2021-08-17    4893.500000
2021-08-18    4911.850098
2021-08-20    4861.299805
2021-08-23    4849.000000
2021-08-24    4890.000000
2021-08-25    4863.450195
2021-08-26    4855.000000
2021-08-27    4846.649902
Name: Open, dtype: float64
```

In [75]:

df_stock

Out[75]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month
Date								
2016-01-01	1154.000000	1165.000000	1145.550049	1162.500000	100178.0	4	1	1
2016-01-04	1161.000000	1173.699951	1137.800049	1149.699951	233101.0	0	4	1
2016-01-05	1147.000000	1157.500000	1138.500000	1147.500000	302272.0	1	5	1
2016-01-06	1149.000000	1154.300049	1135.000000	1137.199951	392164.0	2	6	1
2016-01-07	1115.000000	1136.550049	1115.000000	1126.949951	316523.0	3	7	1
...	...	...	...	...	...	...	...	...
2021-08-23	4849.000000	4888.700195	4788.000000	4874.649902	410817.0	0	23	8
2021-08-24	4890.000000	4912.149902	4792.850098	4863.200195	425873.0	1	24	8
2021-08-25	4863.450195	4875.000000	4818.100098	4856.500000	345800.0	2	25	8
2021-08-26	4855.000000	4877.200195	4829.000000	4846.700195	281990.0	3	26	8
2021-08-27	4846.649902	4934.049805	4824.350098	4910.700195	349435.0	4	27	8

1388 rows × 10 columns

In [76]:

```

history = [x for x in df_stock_train.values]
predictions = list()
for t in range(len(df_stock_test.values[:14])):
    model = ARIMA(history, order=(1,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = df_stock_test.values[t]
    history.append(obs)

```

In [77]:

```
predictions
```

Out[77]:

```
[4881.89654773997,
 4892.4381002807195,
 4927.272614207876,
 5009.311432942523,
 5000.970225379941,
 4954.3878072490825,
 4977.74375063742,
 4935.025094198614,
 4952.725121130176,
 4931.9897013896825,
 4902.817903099244,
 4941.736662759405,
 4897.880080984317,
 4910.500329222549]
```

In [78]:

```
dummy_df['Open'] = predictions.copy()
dummy_df
```

Out[78]:

	Open
Date	
2021-08-02	4881.896548
2021-08-03	4892.438100
2021-08-04	4927.272614
2021-08-05	5009.311433
2021-08-06	5000.970225
2021-08-09	4954.387807
2021-08-10	4977.743751
2021-08-11	4935.025094
2021-08-12	4952.725121
2021-08-13	4931.989701
2021-08-16	4902.817903
2021-08-17	4941.736663
2021-08-18	4897.880081
2021-08-20	4910.500329

In [79]:

```
df_stock_for_test_lin[list(set(variables) & set(list(df_stock.columns)))] = dummy_df.co
py()
```

In [80]:

```
print("Forecasted predictors test data")
df_stock_for_test_lin.head()
```

Forecasted predictors test data

Out[80]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	NIFTY
Date									
2021-08-02	4881.896548	1.003968	1.007218	4928.950195	0.012690	0	2	8	1.0230
2021-08-03	4892.438100	1.017137	1.008719	5006.850098	0.021029	1	3	8	1.0373
2021-08-04	4927.272614	1.022906	1.021318	4964.250000	0.013759	2	4	8	1.0413
2021-08-05	5009.311433	1.017159	1.012530	4923.549805	0.015311	3	5	8	1.0350
2021-08-06	5000.970225	1.003249	1.005024	4916.000000	0.010502	4	6	8	1.0373

In [81]:

```
df_stock_for_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']] = scaler.transform(df_stock_for_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']])
```

In [82]:

```
model_for, variables_for, rmse_per_for = model_generator(df_stock_train_lin, df_stock_for_test_lin)
```

In [83]:

```
print(model_for.summary2())
```

```

Results: Ordinary least squares
=====
Model:                OLS                Adj. R-squared:    0.999
Dependent Variable:    Close              AIC:              13662.6638
Date:                 2021-09-14 14:15    BIC:              13751.4350
No. Observations:     1369              Log-Likelihood:   -6814.3
Df Model:             16                F-statistic:      7.123e+04
Df Residuals:         1352             Prob (F-statistic): 0.00
R-squared:            0.999              Scale:           1248.6
-----
              Coef.    Std.Err.    t      P>|t|    [0.025    0.975]
-----
Open          4382.9999    4.1262  1062.2245  0.0000  4374.9053  4391.0944
Day of Week_0  381.0991    2.1942   173.6846  0.0000   376.7947   385.4035
Day of Week_1  382.2708    2.1760   175.6796  0.0000   378.0021   386.5394
Day of Week_2  382.7929    2.1569   177.4747  0.0000   378.5617   387.0241
Day of Week_3  386.7002    2.1662   178.5114  0.0000   382.4506   390.9498
Day of Week_4  387.8028    2.1856   177.4373  0.0000   383.5154   392.0903
Month_1        153.7430    3.0111    51.0581  0.0000   147.8360   159.6500
Month_2        160.4697    3.1564    50.8395  0.0000   154.2778   166.6617
Month_3        160.3193    3.1042    51.6462  0.0000   154.2298   166.4088
Month_4        163.8607    3.2159    50.9525  0.0000   157.5519   170.1695
Month_5        156.7296    3.0438    51.4916  0.0000   150.7586   162.7007
Month_6        159.8033    3.0557    52.2959  0.0000   153.8087   165.7978
Month_7        167.0932    3.0319    55.1117  0.0000   161.1455   173.0410
Month_8        163.0556    3.3284    48.9887  0.0000   156.5261   169.5850
Month_9        158.9598    3.4005    46.7464  0.0000   152.2890   165.6305
Month_10       160.5114    3.4320    46.7686  0.0000   153.7787   167.2441
Month_11       159.2133    3.3869    47.0087  0.0000   152.5691   165.8574
Month_12       156.9070    3.3837    46.3721  0.0000   150.2692   163.5447
-----
Omnibus:         229.237      Durbin-Watson:     2.239
Prob(Omnibus):   0.000      Jarque-Bera (JB):  2268.361
Skew:           -0.454      Prob(JB):          0.000
Kurtosis:        9.240      Condition No.:     6635984677100544
=====
* The condition number is large (7e+15). This might indicate
strong multicollinearity or other numerical problems.

```

In [85]:

```
print("RMSE Percentage for forecasted predictors",rmse_per_for,sep='\n')
```

```
RMSE Percentage for forecasted predictors
1.1964839437330288
```

In [84]: